Data Science Program, George Washington University

Sample Exam 1

DATS 6103 (Introduction to Data Mining)

Date: October 27, 2026

Instructor: Prof. Sushovan Majhi

Name: _____

Please read the following instructions carefully.

- This is a **sample paper** for practice only, at the real length and the real difficulty, covering everything through Oct 20 but **not** regression as prediction.
- This paper prints **30 problems. Answer any 25.** Each is worth **6 points**, so the exam is marked out of **150**. If you answer more than 25, **your best 25 count**.
- You have **2 hours 30 minutes**, the full session.
- Most problems are multiple choice and **exactly one option is correct**. Mark it clearly (☑). The rest say what they want.
- The exam is **open-book** and **open-note**, but **printed material only**: the chapters covered, the labs whose solutions have been released, the homework covered with its solutions, and your own notes.
- **Laptops**, **tablets**, **phones** and **watches** must be switched off and put away. No AI assistance and no communication of any kind.
- **Nothing has to be executed.** Code you write is graded on whether it would do the right thing, not on a missing colon; a shape, a dtype or the name of an error earns marks.
- All notation has its usual meaning. You may use standard results directly by stating them appropriately. **Read the instructions for individual questions carefully** before answering.

SAMPLE

Question	Points	Score
1	6	
2	6	
3	6	
4	6	
5	6	
6	6	
7	6	
8	6	
9	6	
10	6	
11	6	
12	6	
13	6	
14	6	
15	6	
16	6	
17	6	
18	6	
19	6	
20	6	
21	6	
22	6	
23	6	
24	6	
25	6	
26	6	
27	6	
28	6	
29	6	
30	6	
Total:	180	

SAMPLE

Part A Python

1. (6 points)
```python
a = [1, 2, 3]
b = a
b.append(4)
print(len(a))
```

prints

- ☐ a `TypeError`
- ☐ 3
- ☐ nothing; `append` returns `None`
- √ **4**

2. (6 points)
```python
def push(xs):
    xs = xs + ["b"]

def extend(xs):
    xs.extend(["b"])

p, q = ["a"], ["a"]
push(p); extend(q)
print(p, q)
```

prints

- ☐ `['a', 'b'] ['a', 'b']`
- √ `['a'] ['a', 'b']`
- ☐ `['a'] ['a']`
- ☐ `['a', 'b'] ['a']`

SAMPLE

3. (6 points) `x = [3, 1, 2]` and `y = x.sort()`. What is `y`?

- ☐ `[1, 2, 3]`
- ☐ `[3, 1, 2]`
- ☐ a `TypeError`
- √ `None`

4. (6 points) `d = {"b": 2, "a": 1}`. What does `list(d)` give?

- √ `['b', 'a']`**—the keys in insertion order**
- ☐ an arbitrary order that may differ between runs
- ☐ `[2, 1]`—the values
- ☐ `['a', 'b']`—the keys, sorted

5. (6 points) **Short answer.** This is meant to return a cleaned copy and leave its argument alone. Say what goes wrong, and write a corrected body.

```
def drop_negatives(values):
    for i, v in enumerate(values):
        if v < 0:
            values[i] = 0
    return values
```

Answer. *It edits the caller's list in place and hands back the same object, so the caller's data is changed and* `clean is original` *is* `True`*. Corrected:* `return [0 if v < 0 else v for v in values]`*, or copy first with* `out = list(values)` *and edit* `out`*. Full marks need both the diagnosis and a body that does not touch* `values`*.*

Part B NumPy

6. (6 points) The shape of `np.zeros((4, 1)) + np.zeros((3,))` is
   - ☐ `(4, 1)`
   - ☐ `(3, 4)`
   - ☐ it raises a `ValueError`
   - √ `(4, 3)`

7. (6 points) `a = np.array([4, 5, 6])` and then `a[1] = 9.8`. Afterwards `a[1]` is
   - ☐ `10`—the value is rounded
   - ☐ a `TypeError` is raised
   - √ `9`**—the array is integer, so the value is truncated**
   - ☐ `9.8`—the array is promoted to float

8. (6 points) `A` is a contiguous 2-D array. Which returns a **copy** rather than a view?
   - ☐ `A.reshape(-1)`
   - √ `A[[0, 2]]`
   - ☐ `A.T`
   - ☐ `A[1:3]`

9. (6 points) `T` is `(30, 12, 4)`: days, sensors, channels. `T.mean(axis=1)` returns
   - ☐ a single number
   - ☐ shape `(12, 4)`—one value per sensor per channel
   - ☐ shape `(30, 12)`—one value per day per sensor
   - √ **shape** `(30, 4)`**—one value per day per channel**

SAMPLE

10. (6 points) `k = A.argmax()` for a NumPy array `A`. Which is true of `k`?

    □ it is a `float`

    □ it is a plain Python `int`

    √ **it is a** `np.int64`**, so** `type(k) is int` **is** `False`

    □ it is a one-element array

11. (6 points) `A = np.arange(6).reshape(2, 3)`. Then `A.T.ravel()` is

    √ `[0 3 1 4 2 5]`

    □ `[5 4 3 2 1 0]`

    □ `[0 1 2 3 4 5]`

    □ `[0 3 1 4 2 5]` as a view on `A`

12. (6 points) **Short answer.** Write one line that, for an array `R` of shape `(stores, months)`, divides every value by that store's annual total, leaving the shape unchanged. Say why `R / R.sum(axis=1)` does not do it.

    Answer. *`R / R.sum(axis=1, keepdims=True)`. Without `keepdims` the total has shape `(stores,)`, which right-aligns against the* ***months*** *axis rather than the* ***stores*** *axis: it raises when the two differ and, worse, silently divides by the wrong store's total when they happen to be equal. `keepdims` leaves the total `(stores, 1)`, which broadcasts along the row as intended. `R / R.sum(axis=1)[:, None]` is equally correct.*

Part C Linear algebra in NumPy

13. (6 points) `A = np.array([[2., 0.], [1., 3.]])` and `B = np.array([[1., 4.], [5., 2.]])`. What is `A @ B`?

    √ `[[2, 8], [16, 10]]`

    □ `[[2, 16], [8, 10]]`

    □ `[[3, 4], [6, 5]]`

    □ `[[2, 0], [5, 6]]`

14. (6 points) With the same `A`, the shapes of `A.T @ A` and `A @ A.T` are

    □ both `(15, 15)`

    √ `(4, 4)` **and** `(15, 15)`

    □ `(15, 15)` and `(4, 4)`

    □ both `(4, 4)`

SAMPLE

15. (6 points) If $\text{cond}(A) = 10^4$, then $\text{cond}(A^\top A)$ is about 10^8, which matters because

    - ☐ `solve` will refuse to run
    - ☐ the product is no longer symmetric
    - √ **you lose roughly $\log_{10}$ of the condition number in digits, so the normal equations spend about eight of double precision's sixteen before any arithmetic**
    - ☐ the matrix becomes singular

16. (6 points) You need $\mathbf{x}$ with $A\mathbf{x} = \mathbf{b}$ for one square A and one $\mathbf{b}$. The better call, and why, is

    - ☐ `np.linalg.inv(A) @ b`—it matches the algebra on the page
    - √ `np.linalg.solve(A, b)`**—it factors A once and never forms the inverse**
    - ☐ `np.linalg.pinv(A) @ b`, which is always safer
    - ☐ either; NumPy rewrites one into the other

Part D Pandas

17. (6 points)

```
q1 = pd.Series([100, 120,  90], index=["shaw", "petworth", "navy yard"])
q2 = pd.Series([110,  95, 130], index=["petworth", "navy yard", "brookland"])
total = q1 + q2
```

    `total` has

    - ☐ 3 entries, none missing
    - √ **4 entries, two missing**
    - ☐ 6 entries, none missing
    - ☐ 3 entries, one missing

18. (6 points) `s = pd.Series([10, 20, 30, 40], index=[2, 3, 4, 1])`. Then `s.loc[2]` and `s.iloc[2]` are

    - ☐ both 10
    - ☐ both 30
    - ☐ 30 and 10
    - √ **10 and 30**

SAMPLE

19. (6 points) A group of 10 rows has 3 missing values in column `v`. For that group, `g.size()` and `g["v"].count()` are

    √ **10 and 7—`size` counts rows, `count` counts non-missing values**

    □ 7 and 7

    □ 10 and 10

    □ 7 and 10

20. (6 points) `df[df.score < 0]["score"] = 0` runs without stopping the script. Afterwards `df`

    □ has every score set to 0

    □ has every negative score set to 0

    √ **is unchanged—the assignment landed on a temporary copy, and pandas warns**

    □ has the negative rows deleted

21. (6 points) A column holds `[12.0, 8.0, NaN]`. Its `.mean()` is

    □ 10.0, and `skipna=False` would give the same

    √ **10.0—the gap is skipped and the divisor is 2**

    □ `NaN`, because one value is missing

    □ 6.67—the gap counts as zero

22. (6 points) `orders` has customer `b` once; `lookup` has customer `b` twice. Merging on `cust` produces, for `b`,

    □ 0 rows, because the key is ambiguous

    □ 1 row, keeping the first match

    □ an error

    √ **2 rows, and any total over that column is now double-counted**

23. (6 points) A frame of 20 reviews has 5 missing `rating` values, all of them left by first-time reviewers. Ten of the twenty reviews are by first-timers. After `df.dropna()` the first-timer share is

    □ `0.25`

    √ `0.33`

    □ `0.67`

    □ `0.50`—dropping rows cannot change a proportion

SAMPLE

24. (6 points) `tickets` logs 3 and 5 hours for core in week 1. `pivot_table(index="team", columns="w` gives, for core in w1,

    - □ an error, because the pair is duplicated
    - □ 8, the sum
    - √ **4, the mean, because that is the default** `aggfunc`
    - □ 3, the first value

25. (6 points) **Short answer.** Three orders totalling \$97 are merged against a lookup table in which one customer appears twice. The merged frame has four rows and its `total` sums to \$152. Explain what happened, and name the argument that would have raised instead.

    Answer. *The duplicated key matched one order twice, so that order's row—and its money—appears twice in the result; \$55 is counted a second time, giving* $97 + 55 = 152$*. Nothing raises, because a many-to-one merge is legal. The argument is* `validate="1:1"` *(or* `"m:1"`*). Credit also for the habit: count the rows before and after every join.*

Part E Exploratory analysis

26. (6 points) A histogram of 3,000 rents drawn with five bins looks like a single mound. The safest reading is

    - □ a histogram is the wrong plot for rents
    - □ the rents are unimodal
    - √ **the bin width is a choice, and five bins cannot show structure narrower than a bin**
    - □ the sample is too small to judge

27. (6 points) Mean rent \$2,180, median rent \$1,740. This says the distribution is

    - √ **right-skewed, with a minority of high rents pulling the mean up**
    - □ symmetric, with the difference due to rounding
    - □ left-skewed
    - □ bimodal

28. (6 points) Across 40 schools, average test score and average family income correlate at $r = 0.80$. Within any single school, a pupil's score and their family income correlate at 0.25. Advising one family at one school, the relevant number is

    - √ 0.25**—the only comparison open to that family is with others at their school**
    - □ the average of the two
    - □ neither; correlation says nothing about an individual
    - □ 0.80—it is based on all 40 schools and so is better estimated

SAMPLE

Part F Statistical learning

29. (6 points) A model scores 0.02 mean squared error on its training data and 0.35 on data it has not seen. This is

    √ **overfitting, and the 0.02 is worth nothing as a performance claim**

    □ acceptable, since the average of the two is reasonable

    □ a bug, since test error cannot exceed training error

    □ underfitting, and the model needs more flexibility

30. (6 points) **Short answer.** Of 5,000 transactions, 50 are fraudulent. A model flags none of them. Give its accuracy and its recall on the fraudulent ones, and say in one sentence what you would tell a stakeholder who asks for "the most accurate model".

    Answer. *Accuracy* $4{,}950/5{,}000 = \mathbf{99\%}$*; recall* $0/50 = \mathbf{0}$*. Accuracy averages over classes, and a class that is 1% of the data barely moves it, so a model that has learned nothing scores 99%. The stakeholder should be asked what a missed fraud costs against a false alarm before any metric is chosen—here recall is the target and precision is the budget. Full marks need both numbers and the reason accuracy is uninformative.*

Thirty problems printed; your best twenty-five count.
Code is graded on whether it would do the right thing, not on a missing colon.

SAMPLE